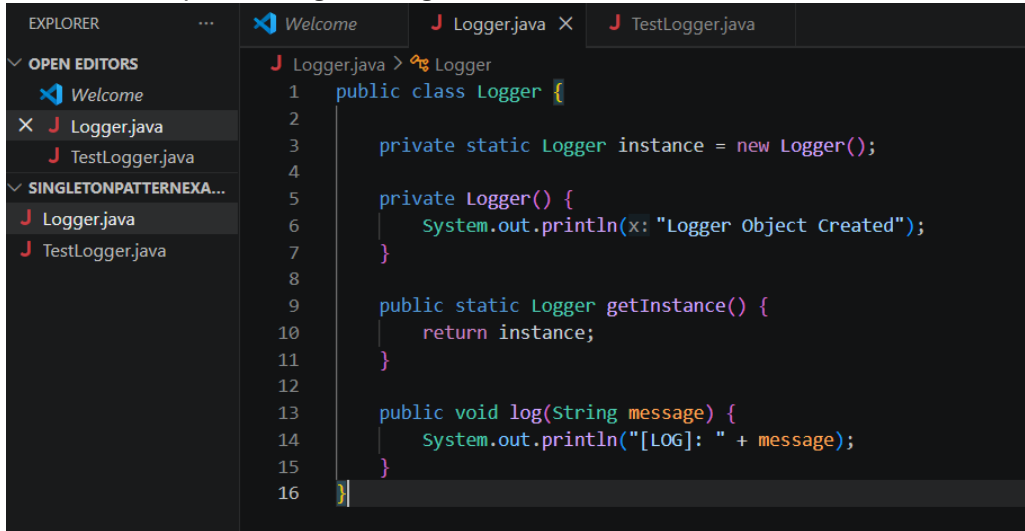


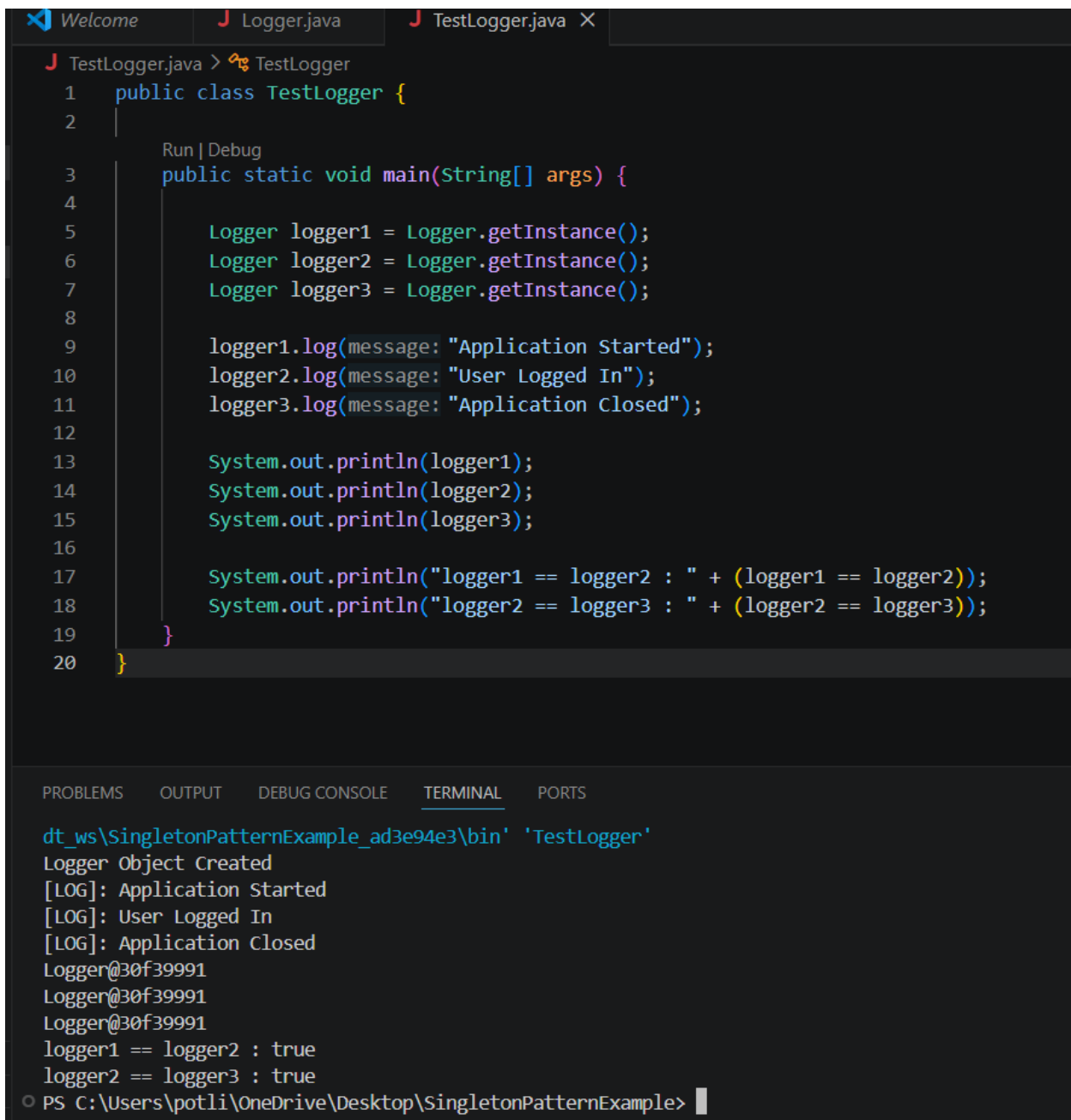
WEEK-1

Design Patterns & Principles

Exercise 1: Implementing the Singleton Pattern



```
1 public class Logger {
2
3     private static Logger instance = new Logger();
4
5     private Logger() {
6         System.out.println(x: "Logger Object Created");
7     }
8
9     public static Logger getInstance() {
10         return instance;
11     }
12
13     public void log(String message) {
14         System.out.println("[LOG]: " + message);
15     }
16 }
```



```
1 public class TestLogger {
2
3     Run | Debug
4     public static void main(String[] args) {
5
6         Logger logger1 = Logger.getInstance();
7         Logger logger2 = Logger.getInstance();
8         Logger logger3 = Logger.getInstance();
9
10        logger1.log(message: "Application Started");
11        logger2.log(message: "User Logged In");
12        logger3.log(message: "Application Closed");
13
14        System.out.println(logger1);
15        System.out.println(logger2);
16        System.out.println(logger3);
17
18        System.out.println("logger1 == logger2 : " + (logger1 == logger2));
19        System.out.println("logger2 == logger3 : " + (logger2 == logger3));
20    }
}
```

dt_ws\SingletonPatternExample_ad3e94e3\bin' 'TestLogger'

Logger Object Created
[LOG]: Application Started
[LOG]: User Logged In
[LOG]: Application Closed
Logger@30f39991
Logger@30f39991
Logger@30f39991
logger1 == logger2 : true
logger2 == logger3 : true

PS C:\Users\potli\OneDrive\Desktop\SingletonPatternExample>

Exercise 2: Implementing the Factory Method Pattern

```
J Document.java > Document
1 public interface Document {
2
3     void open();
4
5 }
```

```
J DocumentFactory.java > DocumentFactory
1 public abstract class DocumentFactory {
2
3     public abstract Document createDocument();
4
5 }
```

```
J WordDocument.java > WordDocument
1 public class WordDocument implements Document {
2
3     @Override
4     public void open() {
5         System.out.println("Opening Word Document");
6     }
7 }
```

```
J WordDocumentFactory.java > WordDocumentFactory
1 public class WordDocumentFactory
2     extends DocumentFactory {
3
4     @Override
5     public Document createDocument() {
6
7         return new WordDocument();
8     }
9
10 }
```

```
J PdfDocument.java > PdfDocument
1 public class PdfDocument implements Document {
2
3     @Override
4     public void open() {
5         System.out.println("Opening PDF Document");
6     }
7 }
```

```
J PdfDocumentFactory.java > PdfDocumentFactory
1 public class PdfDocumentFactory
2     extends DocumentFactory {
3
4     @Override
5     public Document createDocument() {
6
7         return new PdfDocument();
8     }
9
10 }
```

```

ExcelDocument.java > ExcelDocument
1 public class ExcelDocument implements Document {
2
3     @Override
4     public void open() {
5         System.out.println(x: "Opening Excel Document");
6     }
7 }

```

```

ExcelDocumentFactory.java > ExcelDocumentFactory
1 public class ExcelDocumentFactory
2     extends DocumentFactory {
3
4     @Override
5     public Document createDocument() {
6
7         return new ExcelDocument();
8     }
9 }
10

```

```

FactoryTest.java > FactoryTest > main(String[])
1 public class FactoryTest {
2     Run | Debug
3     public static void main(String[] args) {
4         DocumentFactory wordFactory =
5             new WordDocumentFactory();
6         DocumentFactory pdfFactory =
7             new PdfDocumentFactory();
8         DocumentFactory excelFactory =
9             new ExcelDocumentFactory();
10        Document word =
11            wordFactory.createDocument();
12        Document pdf =
13            pdfFactory.createDocument();
14        Document excel =
15            excelFactory.createDocument();
16        word.open();
17        pdf.open();
18        excel.open();
19    }
20 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\potli\
ws\FactoryMethodPattern_52051707\bin' 'FactoryTest'
PS C:\Users\potli\OneDrive\Desktop\FactoryMethodPattern> c::; c
rograms\Eclipse Adoptium\jdk-17.0.10.7-hotspot\bin\java.exe' '
kspaceStorage\798b8e968c8fcad0e34ff6463db9f4a0\redhat.java\jdt
Opening Word Document
Opening PDF Document
Opening Excel Document

```

Data structures and Algorithms

Exercise 2: E-commerce Platform Search Function

```
J SearchTest.java > SearchTest
1 public class SearchTest {
    Run | Debug
2     public static void main(String[] args) {
3         Product[] products = {
4             new Product(productId: 101,
5                 productName: "Laptop",
6                 category: "Electronics"),
7             new Product(productId: 102,
8                 productName: "Phone",
9                 category: "Electronics"),
10            new Product(productId: 103,
11                productName: "Shoes",
12                category: "Fashion"),
13            new Product(productId: 104,
14                productName: "Watch",
15                category: "Accessories"),
16            new Product(productId: 105,
17                productName: "Bag",
18                category: "Fashion")
19        };
20        Product result1 =
21            linearSearch(products, targetId: 104);
22        System.out.println(
23            "Linear Search: " + result1);
24        Product result2 =
25            binarySearch(products, targetId: 104);
26        System.out.println(
27            "Binary Search: " + result2);
28    }
29    public static Product linearSearch(
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\potli\OneDrive\Desktop\ECommerceSearch> c:: cd 'c:\Users\
lipse Adoptium\jdk-17.0.10.7-hotspot\bin\java.exe' '-XX:+ShowCodeDeta
age\f0f7f860f0f1350b13e96e3c729565c8\redhat.java\jdt_ws\ECommerceSear
Linear Search: 104 Watch Accessories
Binary Search: 104 Watch Accessories
PS C:\Users\potli\OneDrive\Desktop\ECommerceSearch> 
```

Exercise 7: Financial Forecasting

```
J FinancialForecast.java > FinancialForecast
```

```
1 public class FinancialForecast {  
2     public static double forecast(  
3         double currentValue,  
4         double growthRate,  
5         int years) {  
6         if(years == 0) {  
7             return currentValue;  
8         }  
9         return forecast(  
10            currentValue,  
11            growthRate,  
12            years - 1)  
13            * (1 + growthRate);  
14    }  
  
Run | Debug  
15    public static void main(String[] args) {  
16        double currentValue = 5000;  
17        double growthRate = 0.10;  
18        int years = 4;  
19        double futureValue =  
20            forecast(  
21                currentValue,  
22                growthRate,  
23                years);  
24        System.out.println(  
25            "Future Value = "  
26            + futureValue);  
27    }
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
C:\Users\potl\AppData\Local\Ecl...  
ws\FinancialForecasting_c8708840\bin' 'FinancialForecast'  
PS C:\Users\potl\OneDrive\Desktop\FinancialForecasting> c:: cd 'c:\Users\potl\n...\nrograms\Eclipse Adoptium\jdk-17.0.10.7-hotspot\bin\java.exe' '-XX:+ShowCodeDetatkspaceStorage\5a9d475802c4c7e821f8483a76a40ccf\redhat.java\jdt_ws\FinancialFore Future Value = 7320.5000000000003  
PS C:\Users\potl\OneDrive\Desktop\FinancialForecasting>
```

PL/SQL

Exercise 1: Control Structures

[SQL Worksheet] *
1 SELECT * FROM Customers;

Query result

Script output

DBMS output

Explain Plan

SQL history

Download

Execution time: 0.012 seconds

	CUSTOMERID	NAME	AGE	BALANCE	ISVIP
1		1 Cheritha	65	15000	FALSE
2		2 Enosh	45	8000	FALSE
3		3 Rainy	70	20000	FALSE

	LOANID	CUSTOMERID	INTERESTRATE	DUEDATE
1	101	1	10	7/9/2026, 6:01:03 P
2	102	2	12	8/8/2026, 6:01:03 P
3	103	3	11	7/14/2026, 6:01:03

Query result

Script output

DBMS output

Explain Plan

SQL history



```
SELECT CustomerID, Age
FROM Customers;...
```

Show more...

Discount applied for Customer 1

Discount applied for Customer 3

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.015

Scenario 2:

Query result

Script output

DBMS output

Explain Plan

SQL history




```

SELECT CustomerID, Balance
FROM Customers;...

```

[Show more...](#)

Customer 1 promoted to VIP

Customer 3 promoted to VIP

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.014

Scenario 3:

```
CURSOR loan_cursor IS
    SELECT c.Name,...
Show more...
```

Reminder: Dear Cheritha, your Loan ID 101 is due on 09-JUL-2026

Reminder: Dear Rainy, your Loan ID 103 is due on 14-JUL-2026

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.014

Exercise 3: Stored Procedures

	ACCOUNTID	ACCOUNTTYPE	BALANCE
1	101	Savings	10100
2	102	Savings	20200
3	103	Current	15000

Monthly Interest Applied Successfully.



SCENARIO 2:

	EMPLOYEEID	EMPLOYEENAME	DEPARTMENT	SALARY
1	1	John	IT	55000
2	2	David	HR	40000
3	3	Smith	IT	66000

Bonus Updated Successfully.

SCENARIO 3:

	ACCOUNTID	ACCOUNTTYPE	BALANCE
1	101	Savings	9600
2	102	Savings	20700
3	103	Current	15000

Transfer Successful.